Milestone C Journal

Akshith Gunasheelan
agunasheelan3@gatech.edu

1 INTRODUCTION

The main goal of this milestone is to ensure the ARC-AGI agent addresses the problems in problem set C. These problems require the agent to identify relationships between input and output and apply transformations that generalize across examples. The problems in set C require more complex and structured reasoning compared to previous milestones.

My approach builds on the rule-based approach which was followed in the earlier milestones. The agent infers a transformation rule that explains the mapping between input and output examples in the training set. Once a rule is identified, the rule is applied to all training examples. If it succeeds, it is applied to the test input to generate the prediction.

By evaluating these rules systematically against the given training examples, the agent selects the first rule that explains the observed input-output relationships correctly. This is the basic implementation design.

2 AGENT DESIGN AND WORKING

For every problem the agent receives a set of input and output pairs and single test input. For each of the defined rules the agent performs the following operations –

1. Rule is applied to every example in the training set.
2. Result of the above step is compared with the corresponding training output grid.
3. If the rule produces correct output for every training example in the set, the rule is considered to be correct for that problem.
4. Once the correct rule is identified, it is applied to the test input to produce the predicted output.

This process finally selects the first rule that can explain all the training examples consistently. The rules are also evaluated sequentially in a predefined order in a python list, enabling the agent to apply simpler transformations before applying

complex ones. The transformation rules implemented in the agent include several common patterns that are found in ARC problem prediction. Some

1. **Grid extraction and cropping transformations** – These problems identify the region of the grid by removing redundant cells and extracting the bounding box the contains the required structure. Figure 1 shows an example of a problem in this category. The agent detects 4 marker pixels and interprets this as the boundaries. The grid / pixels inside this boundary is extracted and their colour is changed to form the output grid. My code implements a *replace_and_exchange()* function to achieve this. Each cell in the grid is processed at most a few times, the time complexity of this rule is **O(mn)**, where m and n represent the grid dimensions. The space complexity is also **O(mn)** due to storing intermediate grids during transformation.

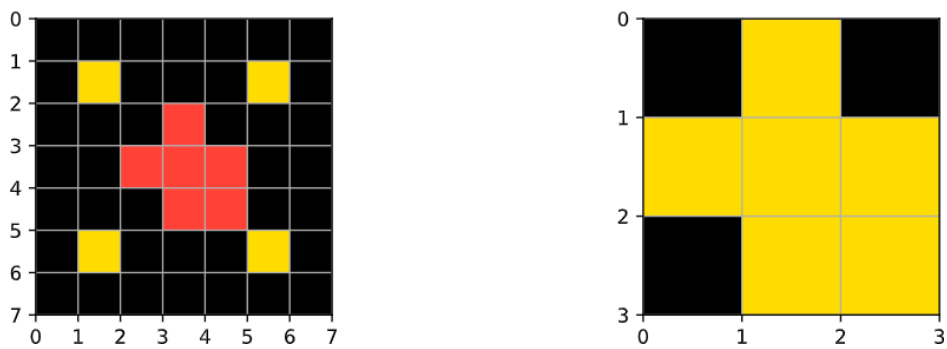

Figure 1 – This shows the example 3de23699 from problem set C of ARC project.

2. **Pattern expansion/ Geometric transformations** - These problems read the input figure and transforms the pixels into some meaningful output grid by building on top of the input. Figure 2 shows such an example, where the input grid contains 2 coloured objects, whose corners are extended outward toward the boundary of the grid. The *corner_and_expand()* function detects these corner positions and propagates the colors along diagonal directions to form the output pattern. Even in this problem the algorithm scans the entire grid so time complexity is **O(mn)** and space complexity is **O(mn)** for an m x n grid.

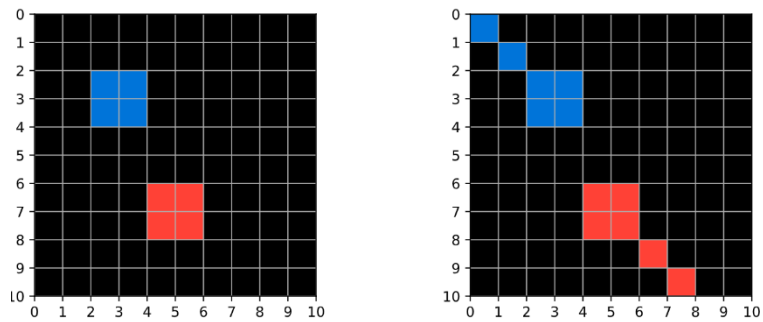

Figure 2 – This shows the example 5c0a986e from problem set C of ARC project.

3. **Mirroring or Symmetric transformations** – In these types of questions usually there is some kind of mirror or symmetric transformations applied to get a result. Figure 3 shows input grid , where it is expanded by reflecting its pattern horizontally and vertically to create a larger symmetric grid. The function *mirror_and_fake()* first mirrors the grid left to right, concatenates the result, and then mirrors the combined grid vertically to form a symmetric pattern. Since the transformation processes each cell constant number of times, the time complexity is **O(mn)** and the space complexity is **O(mn)** for an $m \times n$ grid.

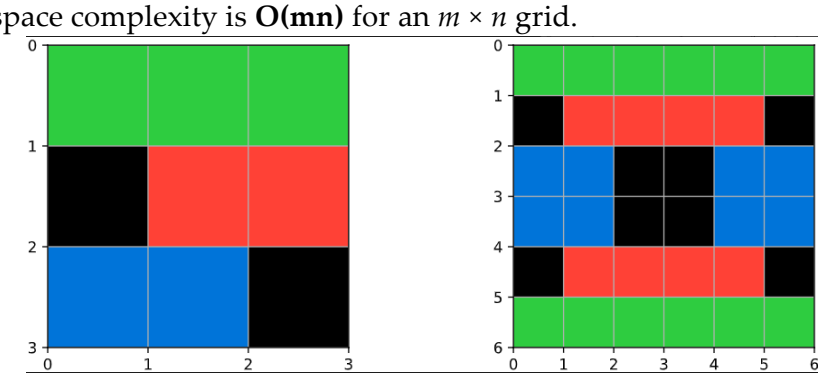

Figure 3 – This shows the example 62c24649 from problem set C of ARC project.

In addition to these examples discussed above, this agent has used several other transformation function for solving problems. Some of these transformation functions include color replacement, filling in hollow regions, color frequency reconstruction, rotation, etc. The agent can make up to three predictions for a problem, as required by the project requirement. However, in my agent, in most cases, only a single rule is sufficient for explaining all the training examples, so my agent tends to make only one prediction.

3 AGENT PERFORMANCE

The agent was tested on the set C problems, which included 16 training problems and 16 test problems.

Training C Problems Solved: 8 / 16

Test C Problems Solved: 8/ 16

During the development phase, the agent was tested using eight transformation rules that had been implemented. It was observed that the transformation rules were successful in solving all eight of the training problems, as required by the milestone for full marks. The agent is effective in solving problems that have one of the implemented transformation rules. If the rule is successful in explaining the training examples, the rule can be used to produce the correct output for the test input.

3.1 Problems the agent solves well

The agent performs well on ARC problems in which the transformation can be defined by a well-defined rule that is applicable to all the training examples. These include problems which were discussed in the performance section such as grid extraction, geometric transformations like mirroring or rotation, colour replacement patterns and expansion of patterns in which structures are expanded diagonally or symmetrically.

3.2 Problems where the agent struggles

The current agent faces difficulties in solving problems that involve multiple independent objects in the grid. In such cases, the correct transformation may involve the identification of connected components in the grid.

The agent is also facing difficulties in solving problems that involve multiple transformations in sequence. This is because, in the current implementation, it is assumed that only one rule is enough to explain the relationship between the input and output grids.

Figure 4 shows an example of complex reasoning. The problem of filling the hollow areas in objects, which is being solved by the agent. This problem involves

using a breadth-first search strategy to identify enclosed areas in the objects before applying the transformation and has many corner cases as well.

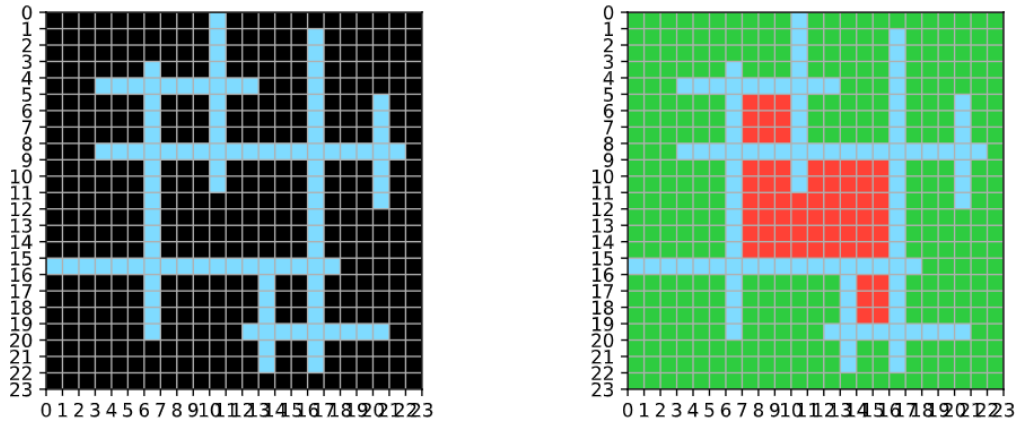

Figure 4 – This shows the example 7b6016b9 from problem set C of ARC project.

4 AGENT EFFICIENCY

The agent evaluates a fixed number of transformation rules for each problem. Most rules require scanning the grid a few numbers of times, resulting in a time complexity of O(mn) for a grid of size m × n.

Since ARC grids are relatively small in the current milestone, the cost of evaluating these rules isnot very high. The agent usually solves each problem in well under one second.

The space complexity is also O(mn), as intermediate grids may be created during the transformation process.

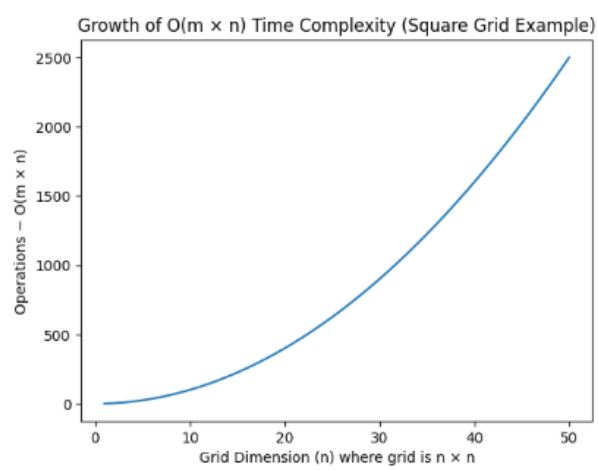

Figure 5 – This figure shows the time complexity analysis for a m X n grid

Figure 5 illustrates the growth of the $O(m \times n)$time complexity used by several transformation rules in the agent. Each rule scans the grid a constant number of times, the number of operations increases proportionally to the total number of cells in the grid. For square grids where $m = n$, this corresponds to $O(n^2)$growth.

5 FUTURE IMPROVEMENTS

However, there are certain changes that can be made to enhance the performance of the agent in solving more complex ARC problems. One such change would be the inclusion of object detection methods to identify the connected components in the grid and apply transformations to the objects individually. Another change would be the inclusion of the ability for the agent to apply multiple transformations in order rather than relying on a single rule to solve the entire problem. Another change would be the extension of the rule set for transformations.

6 EXPECTED PERFORMANCE ON SET D

Based on the current design, the agent is expected to perform well on Set D problems that involve geometric transformations, color replacement, cropping operations, and symmetric pattern generation. However, problems that require reasoning about multiple objects or applying several transformations sequentially may remain challenging for the current implementation.

7 FEEDBACK FROM PEERS

Feedback from classmates would be particularly helpful in identifying additional transformation patterns that commonly appear in ARC problems. Suggestions on approaches for object detection or techniques for combining multiple transformations would be valuable in improving the robustness of the agent. Also I was finding generalizing difficult even though I have implemented many helper functions which are common to many problems. But still, I feel if there is a new unseen problem, my agent will struggle unless it has some code to identify the pattern. To fully make it like a smart model will require some additional intelligence. I would like to know if anyone has some ideas for such implementations, thank you.